

Developer Operations (devops)

Cloud Efficiency



Commercial Break

CS-Seminar:

Lena Fuhrmann:

Level Up Your Serverless Game

Today!!!

”Studienrichtung”:

Security, Platforms and DevOps

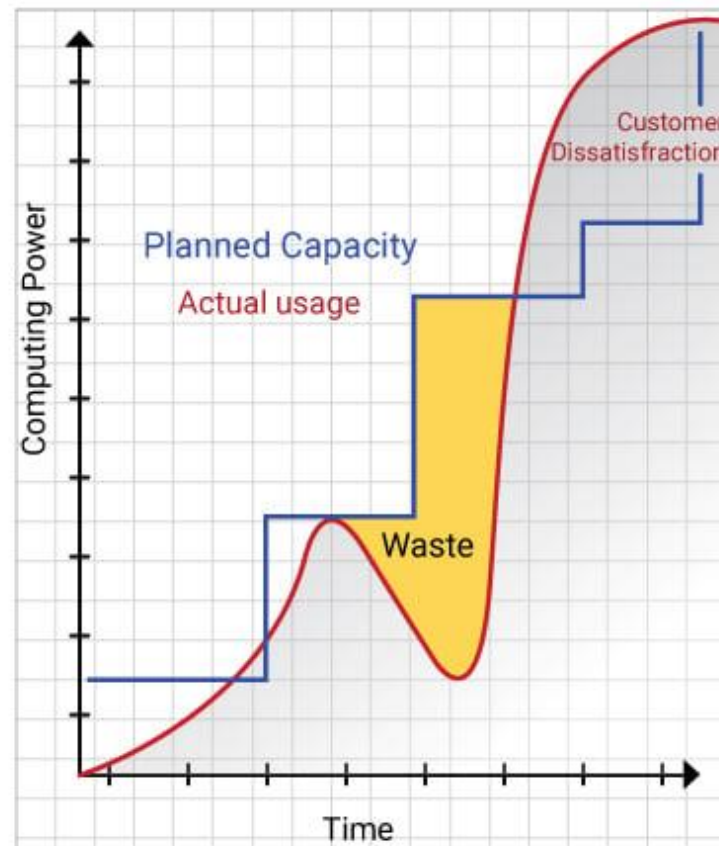
<https://www.fhnw.ch/de/studium/informatik/security-platform-devops>

→ Starting FS26

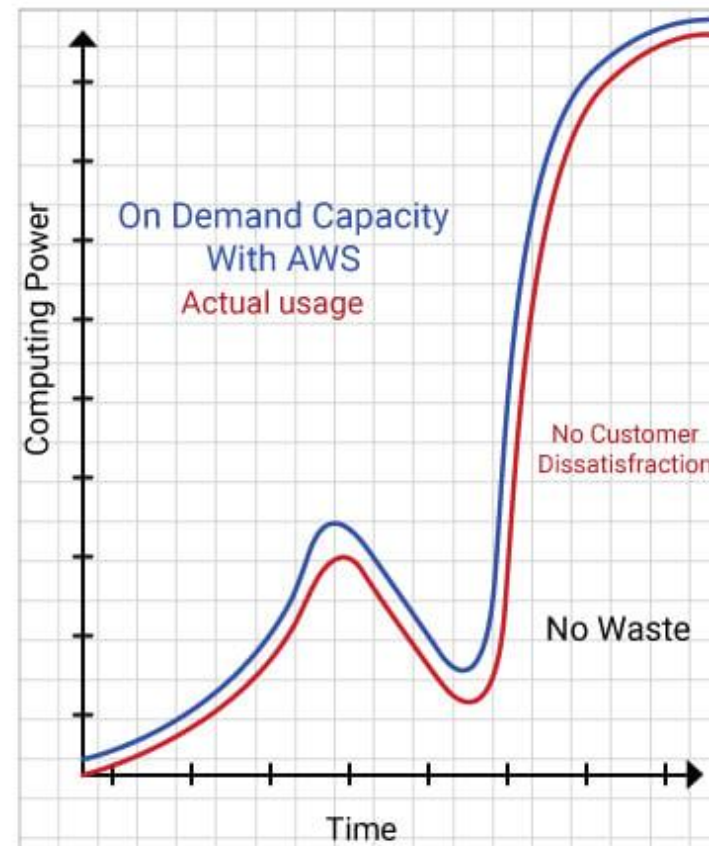
Help wanted: If you are interested to support the launch, we are currently looking for students for some trailer / small movies

Cost and Workload

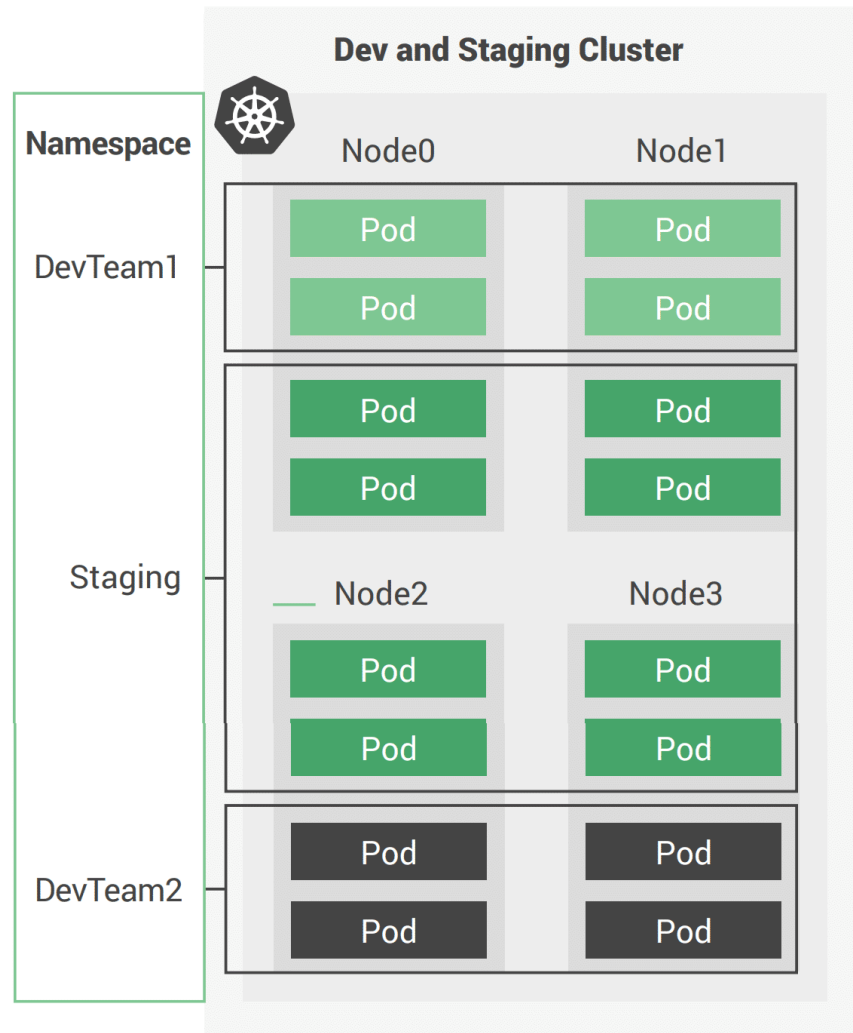
Traditional Data Center



Amazon Web Services



Container Platforms, Pay as you go



Based on the platform:

- Size of Pod affects scheduling on node
- Scheduling on nodes results in scaling in / scaling out of compute nodes / compute resources

→ Sizing of pods directly affect costs

Costs of infrastructure

“Memory is trickier to calculate than cost per core but we can set it at **\$25 per 8 GB per month**. We can look at [Google Cloud](#), which allows you to flexibly combine different vCPU and memory amounts to build a machine. Their costs come in at \$2.40 per GB for the e5 series.

On AWS, we can look at the step up in memory for machines with the same vCPU count.

A [c5.2xlarge](#) has 8 vCPUs and 16 GB of memory while a [m5.2xlarge](#) has 8 vCPUs with 32 GB of memory. The price jump for an extra 16 GB memory? \$32 per month, or close to the same cost per GB of memory as GCP.”

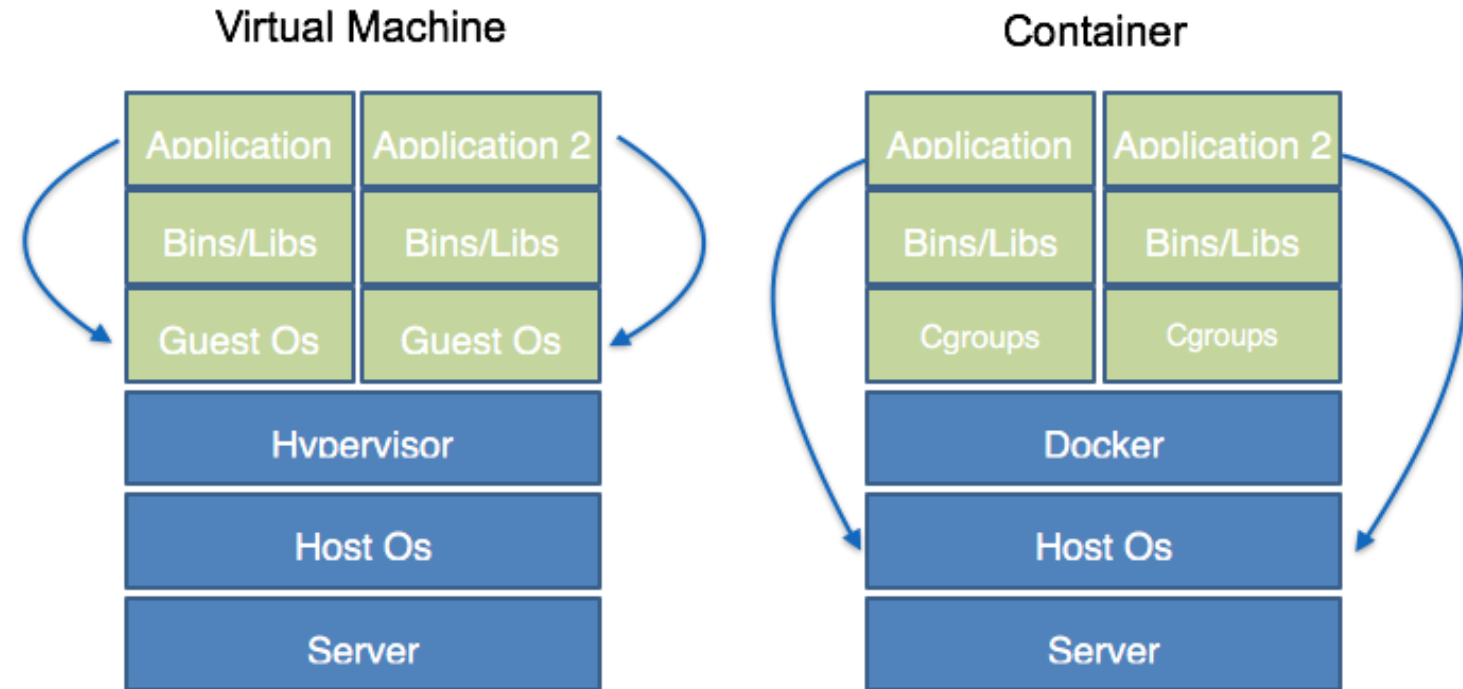
<https://www.vantage.sh/blog/cloud-costs-every-programmer-should-know#cost-estimation-references>

Resource	Monthly Cost
10 GB data inter AZ	\$0.10
10 GB blob storage	\$0.50
10 GB attached storage	\$0.80
10 GB data egress	\$0.90
10 database IOPS	\$1.00
10 custom metrics	\$2.00
1 IPv4 address	\$4.00
10 GB logs	\$5.00
10 GB memory	\$25.00
1 vCPU	\$30.00
Redis server	\$160.00
Postgres server	\$180.00
1 GPU (A100)	\$3,000.00
1 GPU (H100)	\$9,000.00

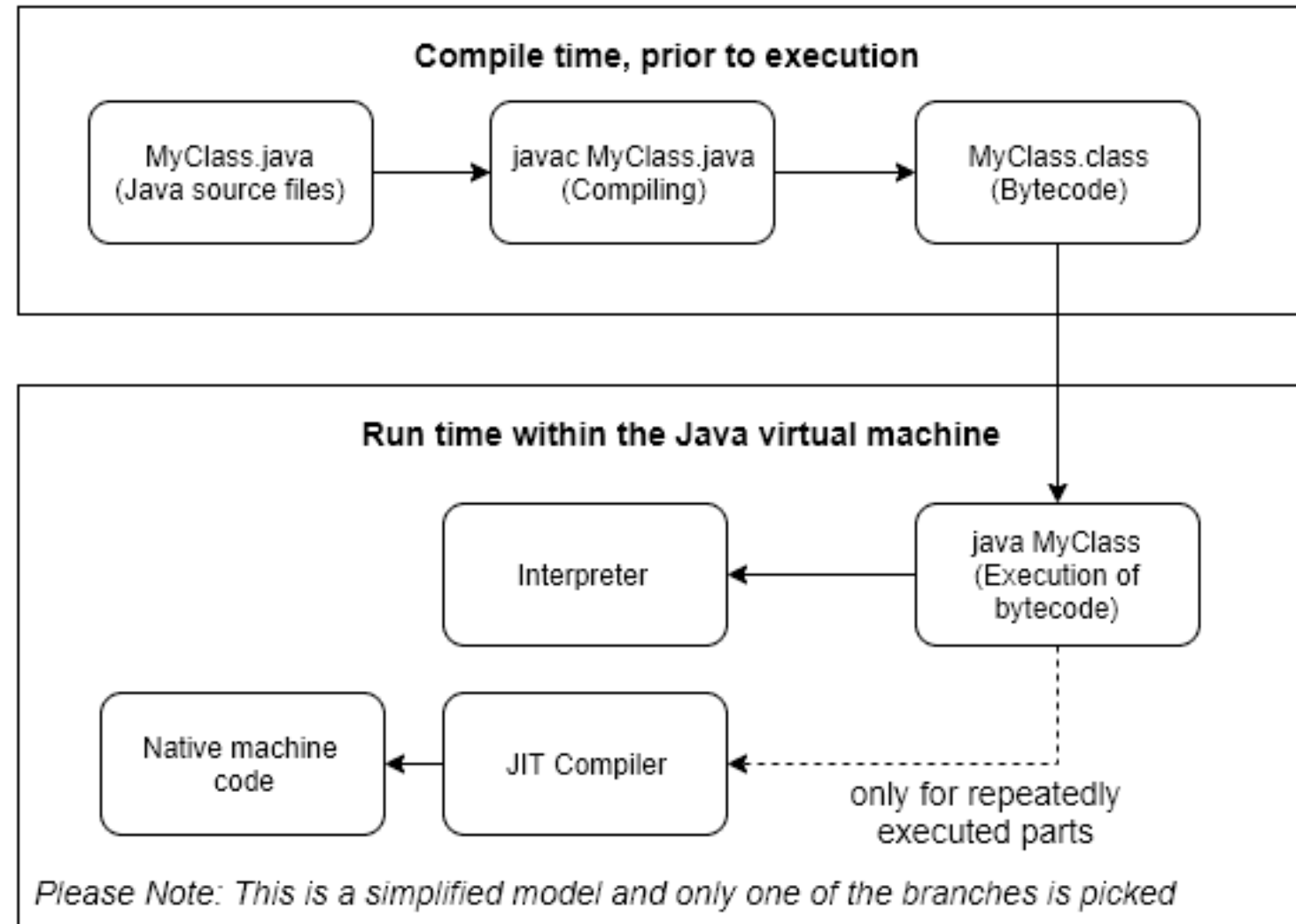
Excuse: Java and the Cloud?

JVM is an own “virtual machine”

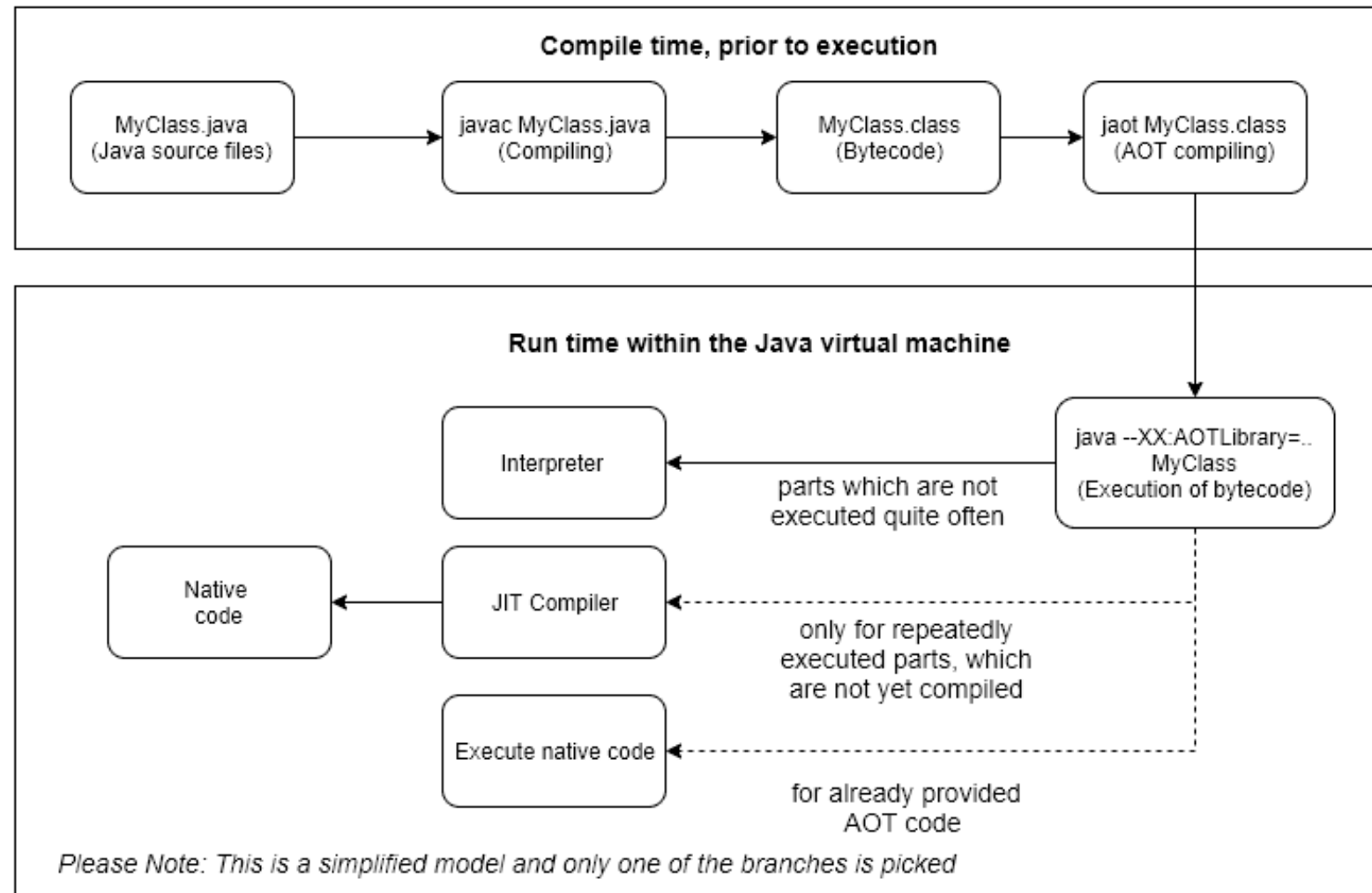
- JVM adjusts to the number of processors
 - Compiler Threads
 - Garbage Collector
 - Size of Fork-Join Pool
- No Support for cgroups until JDK9
- Main Problem stays:
 - If not defined: containerized JVMs do not know each other → no synergies between containers



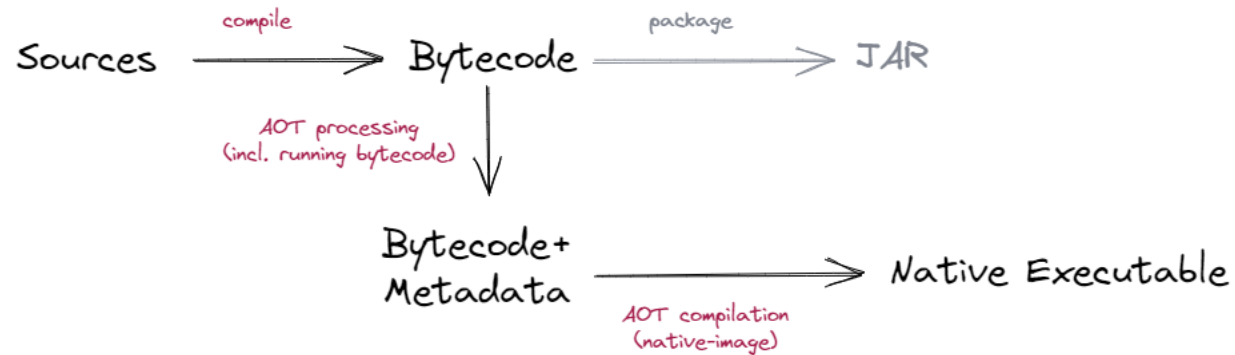
What about Java?



Java and Native?



Native Images with GraalVM



+:

- Instant Startup Time
- Low Memory Consumption

-:

- Slower builds because of AOT
- No Reflection / Dynamic Proxies / Serialization / JNI (even java.util.Random)
- No JVM-Tools

Solutions: Specialized JVM (GraalVM) and Framework (Quarkus)

Runtime Mode	Memory (MB)	Number processes per 2048MB	Max Throughput per Process (Req/Sec)	Overall Max Throughput (Req/Sec)	Compared to Thorntail
Thorntail	651	3	37,082	111,246	100%
Quarkus - JVM	414	4	77,118	308,472	277%
Quarkus - Native	122	16	44,841	717,456	645%

<https://quarkus.io/blog/runtime-performance/>

Computation example

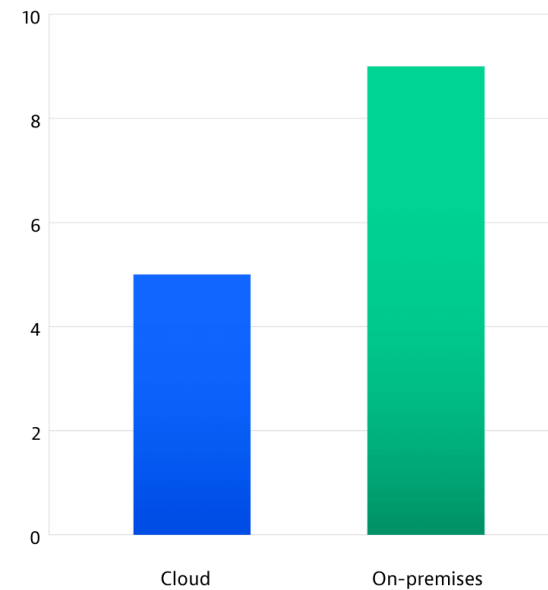
Memory Footprint, Java:

- JVM: 414MB
- Native: 122MB

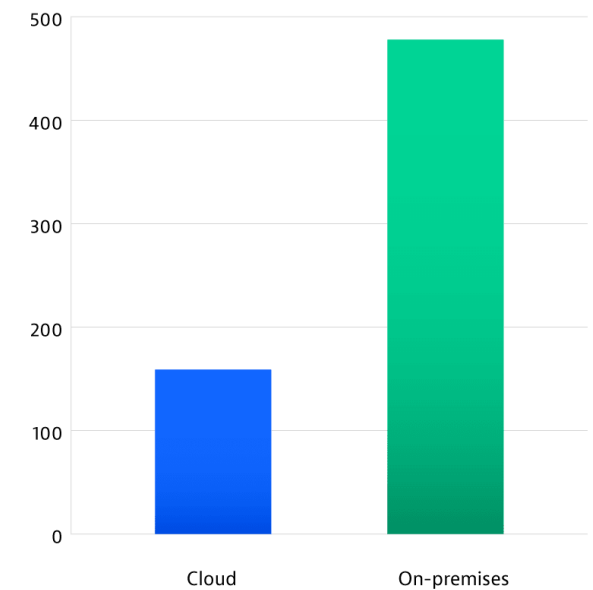
Example:

- Pods per Cluster: 300
- Price per GB: 2.5\$
- Optimal Scheduling

Nodes per cluster
(median)



Pods per cluster
(median)



<https://www.dynatrace.com/news/blog/kubernetes-in-the-wild-2023/>

“Modern” languages

Different target functions to optimize to:

- In “normal” workloads (12factor-services), compute is seldomly the constraint and rather constant
 - Apps get slow but will not stop with too less CPU
- Memory is fix

<https://greenlab.di.uminho.pt/wp-content/uploads/2017/09/paperSLE.pdf>

	Mb
(c) Pascal	1.00
(c) Go	1.05
(c) C	1.17
(c) Fortran	1.24
(c) C++	1.34
(c) Ada	1.47
(c) Rust	1.54
(v) Lisp	1.92
(c) Haskell	2.45
(i) PHP	2.57
(c) Swift	2.71
(i) Python	2.80
(c) Ocaml	2.82
(v) C#	2.85
(i) Hack	3.34
(v) Racket	3.52
(i) Ruby	3.97
(c) Chapel	4.00
(v) F#	4.25
(i) JavaScript	4.59
(i) TypeScript	4.69
(v) Java	6.01
(i) Perl	6.62
(i) Lua	6.72
(v) Erlang	7.20
(i) Dart	8.64
(i) Jruby	19.84

